Contents

Volume 04 — GANs (L31–L43) **1**
L31: Motivation for GANs . . . 1
L32: GAN architecture . . . 2
L33: GAN objective and loss functions . . . 2
L34: GAN Convergence and Nash Equilibrium . . . 3
L35: Introduction to DCGAN . . . 4
L36: Practical Exercise 1 — Vanilla GAN . . . 5
L37: Practical Exercise 2 — DCGAN . . . 6
L38: Conditional GAN . . . 7
L39: Pix2Pix GAN . . . 8
L40: CycleGAN . . . 8
L41: StyleGAN . . . 9
L42: StyleGAN 2 . . . 10
L43: Practical exercise 1 — CycleGAN (advanced GAN lab) . . . 11

Volume 04 — GANs (L31–L43)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L31: Motivation for GANs

Learning focus

- List the **six VAE limitations** this lecture uses as the reason to leave explicit likelihoods.
- State the GAN **motivation** in one sentence: realistic samples **without** a prescribed $p(x)$ or reconstruction loss.
- Define **adversarial learning** as two networks that compete: **generator** vs **discriminator**.

Exam lines

- VAEs are limited by **Gaussian posterior/prior**, **pixel MSE averaging** (3 vs 8 with similar z), **rec↔KL trade-off**, **posterior collapse**, **likelihood choice**, and **missing fine texture**.
- GAN motivation: realistic samples **without** writing $p(x)$ or a reconstruction term.
- GAN = two nets: $G(z) \rightarrow$ fakes, D scores real vs fake; they **compete** and G trains on D's feedback.

Concept tables

Piece	Assumption
Approximate **posterior** $q(z \mid x)$	**Gaussian**, so the encoder only emits μ and $\log \sigma^2$
Prior $p(z)$	**standard normal** $\mathcal{N}(0, I)$
Data type	Decoder emits
———–	———
Continuous	means
Binary	per-bit probabilities

L32: GAN architecture

Learning focus

- Trace **generator training**: $z \to G_\phi(z) = \hat{x} \to D_\theta(\hat{x}) = \hat{y}$, BCE with **target** $y = 1$, update **only** ϕ.
- Trace **discriminator training**: real branch ($y = 1$) and fake branch ($y = 0$), $L_D = L_{\text{real}} + L_{\text{fake}}$, update **only** θ.
- Derive both losses from **binary cross-entropy** and write the chain-rule updates.
- Reproduce the lecture's probability examples (0.2 vs 0.9 / 0.8).

Exam lines

- G_ϕ: Gaussian $z \to$ image $\hat{x}$ **matching real shape** $\to D_\theta(\hat{x})$; $L_G = -\log D_\theta(G_\phi(z))$ with $y = 1$; update ϕ, freeze θ.
- D_θ: real x wants score ≈ 1 ($y = 1$); fake $G_\phi(z)$ wants score ≈ 0 ($y = 0$); $L_D = L_{\text{real}} + L_{\text{fake}}$; update θ, freeze ϕ.
- Same BCE, different targets, **alternating** freezes. Next lecture turns this into a **min / max** objective.

Concept tables

Who is training	Input to D	Target y
Generator	fake $\hat{x}$	**1** (fool D)
Discriminator	real x	**1**
Discriminator	fake $\hat{x}$	**0**

L33: GAN objective and loss functions

Learning focus

- Re-derive L_{real}, L_{fake}, and L_D from BCE (same algebra as Lec 32).
- Turn **minimize** L_D into **maximize** a value function V_D, then write **expectations** for minibatches.

- Explain **why** $\max_\theta V$ pushes $D_\theta(x) \to 1$ and $D_\theta(G_\phi(z)) \to 0$, using the lecture's $\log 0.1 / \log 0.5 / \log 0.9$ table.
- Write the **generator** value function (optimize **only the fake term**) and the combined **minimax** GAN objective.

Exam lines

- $L_D = -\log D(x) - \log(1 - D(G(z)))$ on one pair; **min** L_D = **max** V_D with the minus dropped.
- Batches replace single logs by **expectations** over p_{data} and p_z.
- $\max V_D$ raises $\log D(x)$ ($D(x) \to 1$) and $\log(1 - D(G(z)))$ ($D(G(z)) \to 0$); the $0.1/0.5/0.9$ log table is the proof.
- G cannot touch the real term; it **minimizes** $\log(1 - D(G(z)))$, i.e. wants $D(G(z)) \to 1$ (and hits $\log 0$ if it fully succeeds).
- Full GAN: $\min_G \max_D V(G, D)$ — two opposite objectives on the same V.

Concept tables

$D_\theta(x)$	$\log D_\theta(x)$ (approx.)	Reading
0.1 (untrained, poor)	-2.303	D fails on reals
0.5 after some training	-0.693	improving
0.9 later	-0.105	near the ceiling
$D_\theta(G_\phi(z))$	$1 - D$	$\log(1 - D)$
———————:	——:	————:
0.9 (bad D)	0.1	-2.303
0.5	0.5	-0.693
0.1 (good D)	0.9	-0.105

L34: GAN Convergence and Nash Equilibrium

Learning focus

- State what $p_{\text{data}}(x)$ and $p_g(x)$ are, and how generated samples are produced from z.
- Describe the **start of training**: noisy G, a strong D, and $p_g \neq p_{\text{data}}$.
- Track how discriminator feedback moves p_g toward p_{data} and $D(G(z))$ toward 0.5.
- Define **GAN convergence** and **Nash equilibrium** in the lecture's two-player language.

Exam lines

- Real data: $x \sim p_{\text{data}}(x)$. Fakes: $z \sim \mathcal{N}$, then $G_\phi(z)$ with collection $p_g(x)$.
- At init, $p_g \neq p_{\text{data}}$; D scores reals near 1 and fakes near 0.
- Training updates ϕ from D's feedback; p_g moves toward p_{data} and $D(\text{fake})$ climbs toward 0.5.

- **Convergence:** $p_g = p_{\text{data}}$ and $D \equiv 0.5$ on both. Same state is **Nash**: neither player improves by changing only its own weights.

Concept tables

Source	What the images look like at initialization
Real $x \sim p_{\text{data}}$	Actual faces
Fake $G_\phi(z)$	Noise / disturbed / unrealistic blobs
Input	Discriminator output (probability "real")
——-	————————————————-
Real face	closer to 1
Generated blob	closer to 0

L35: Introduction to DCGAN

Learning focus

- Recap vanilla GAN as two networks with **opposite** (minimax) objectives.
- List four **limitations** of GANs taught here: no convergence guarantee, mode collapse, vanishing gradients, hyperparameter sensitivity.
- State the **DCGAN design rules** (strided / transposed conv, batch norm, no FC, activations).
- Sketch the lecture's **64×64 RGB** generator and discriminator tensor sizes.

Exam lines

- Vanilla GAN is a minimax game; it can fail to converge, collapse modes, starve G of gradients, and blow up if you retune mid-run.
- DCGAN: strided conv in D, transposed conv in G, BN both sides, drop FC, ReLU/tanh in G, Leaky ReLU/sigmoid in D.
- Lecture's picture: 100-D $z \to 4 \times 4 \times 1024 \to 64 \times 64 \times 3$; D mirrors that down to $4 \times 4 \times 512$ then sigmoid.
- StyleGAN / BigGAN / WGAN are taught as **add-ons on the DCGAN foundation**, not replacements of that core.

Concept tables

Network	Input	Job
Generator G	latent noise z	Emit images that D will accept as real
Discriminator D	real images **and** G's images	Tell real from generated
Piece	Vanilla GAN	DCGAN

Network	Input	Job
——-	——--	——–
Discriminator spatial ops	pooling	**strided convolution**
Generator spatial ops	(dense upsample)	**transposed convolution** (also called deconvolution / **fractionally strided** convolution)
Batch normalization	not used	**BN in G and D**
Fully connected layers	present	**removed** in deeper architectures
G activations	—	**ReLU** hidden; **tanh** at output
D activations	—	**Leaky ReLU**; **sigmoid** at last layer (real vs fake: 0 or 1)

L36: Practical Exercise 1 — Vanilla GAN

Learning focus

- Implement a **vanilla GAN** in TensorFlow/Keras on **Fashion MNIST**.
- Normalize pixels to $[-1, 1]$ to match **tanh**, flatten $28 \times 28 \to 784$, and drop labels / test split.
- Build MLP **generator** and **discriminator**, a custom `GAN` model class, **Adam + binary cross-entropy**, and plot both losses.
- Read the loss curves and the 4×4 grid of generated garments.

Exam lines

- Unsupervised Fashion MNIST: **60k × 784**, pixels in $[-1, 1]$, no labels/test.
- G: $100 \to 256 \to 512 \to 1024 \to 784$ tanh, Leaky ReLU 0.2; D is the reverse plus sigmoid.
- Train with a Keras `GAN` class, two Adams ($\beta = 0.5$), BCE, optional 0.05 label noise, misleading ones for G.
- Early: D loss $\ll$ G loss; later they track; 16 grayscale samples already look like garments.

Concept tables

Network	Role in this lab
Generator	Noise $z \rightarrow$ **fake** 28×28 fashion image
Discriminator	Classifier: **real** Fashion MNIST vs **fake**

Class	Item
—:	——
0	T-shirt / top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

L37: Practical Exercise 2 — DCGAN

Learning focus

- Explain why **convolutions** beat the Lec 36 MLP: spatial structure, feature hierarchy, fewer parameters.
- Rebuild Fashion MNIST as $28 \times 28 \times 1$, with shuffle/batch pipeline, `Conv2DTranspose` generator and strided-conv discriminator.
- Reproduce BN + `use_bias=False`, padding, stride-2, Leaky ReLU 0.2, dropout 0.3, Adam 2×10^{-4}, $\beta = 0.5$, BCE **`from_logits=True`**.
- Compare sample quality and param counts to vanilla GAN; note the missing **class control** (teaser for cGAN).

Exam lines

- Keep Fashion MNIST as $28 \times 28 \times 1$; shuffle 60k; batch 128; $z \in \mathbb{R}^{100}$.
- G: dense $7 \times 7 \times 256$ (no bias) $\rightarrow$ Conv2DTranspose + BN + ReLU $\rightarrow$ tanh; $2\,330\,944$ params.
- D: strided conv 64 then 128, Leaky ReLU 0.2, dropout 0.3, dense 1; $212\,865$ params.
- Adam 2×10^{-4}, $\beta = 0.5$, BCE with logits; 20 epochs end near $G\,0.81$, $D\,1.32$.
- Better images than vanilla GAN, still **unconditional** — that is the hook for Lec 38.

Concept tables

Vanilla GAN	DCGAN
Fully connected	**Convolutional**
Spatial structure discarded	Spatial structure **preserved** (padding helps keep borders)

Vanilla GAN	DCGAN
No explicit hierarchy	**Low-level** (edges, corners) → **mid-level** (textures, shapes) → **high-level** (object parts)
More parameters	**Fewer** parameters, less memory, faster training
Knob	Value
——	——:
Buffer / training size	**60 000**
Batch	**128**

L38: Conditional GAN

Learning focus

- Restate vanilla GAN as a **minimax / cricket-style** adversarial game, and why z alone cannot request "digit 5."
- Draw the **cGAN** architecture: condition y into **both** G and D.
- Write **real / fake** D **losses**, the **non-saturating** G **loss**, and the **conditioned minimax** $V(D, G)$.
- Contrast vanilla vs conditional GAN and list the lecture's applications (including the Pix2Pix teaser).

Exam lines

- Vanilla z has no semantic map to classes; you cannot request "digit 5."
- cGAN feeds y to **both** G and D; $G(z, y)$ is the controlled fake.
- $D(x, y) \to 1$, $D(G(z, y), y) \to 0$ when training D; G pushes the fake score to 1.
- Practical $L_G = -\mathbb{E}[\log D(G(z, y), y)]$; objective is minimax V **conditioned on** y.
- Same game as vanilla GAN, plus **label-consistent** (or text/image-conditioned) samples.

Concept tables

Sample	$G(z)$ might be
z_1	digit **3**
z_2	digit **7**
z_3	digit **1**
$D(x, y)$	$\log D(x, y)$
——:	——:
0.9	≈ -0.045
0.2	≈ -0.70

L39: Pix2Pix GAN

Learning focus

- Place Pix2Pix as a **paired image-to-image** specialist of **cGAN**.
- Explain why the generator is a **U-Net** (skip connections) and the discriminator a **PatchGAN**, including the 70×70 patch result.
- Write G**'s loss** $L_{\text{cGAN}} + \lambda L_1$ with $\lambda = 100$, and D's real+fake BCE.
- State training details (alternating Adam steps) and that D **is dropped at inference**.

Exam lines

- Paired translation: sketch↔photo, map↔satellite, LR↔HR, inpainting-style repair.
- U-Net skips restore spatial detail the encoder would drop.
- PatchGAN averages $n \times n$ real/fake votes; paper's best patch 70×70.
- $L_G = L_{\text{cGAN}} + 100\, L_1$; L_D = real ones + fake zeros; alternate Adam 2×10^{-4}.
- Inference = generator only.

Concept tables

Input	Output
Edges / **sketch**	**Photo**
Map	**Satellite** image
Sketch	**Painting**
Low resolution	**Super-resolution**
Damaged image	**Repaired** reconstruction
	Conditional GAN
–	———————–
Scope	**Generic** conditional generation
Condition y	labels, **text**, **images**, …
Generator	ordinary net (as in Lec 38)
Discriminator	ordinary net

L40: CycleGAN

Learning focus

- Explain the **paired-data** bottleneck of Pix2Pix and why **unpaired** translation needs CycleGAN.
- Name the **four networks** G, F, D_Y, D_X on domains X (horse) and Y (zebra).
- State **cycle consistency**: horse→zebra→horse returns the **same** horse (and the backward cycle).
- Write the two **adversarial** losses, L_1 **cycle** loss, $\lambda = 10$, and the joint minimax objective.

Exam lines

- Unpaired $X \leftrightarrow Y$: no matched sketch/photo required.
- $G : X \to Y$, $F : Y \to X$, D_Y judges Y, D_X judges X.
- Cycle consistency: reconstruct the **same** horse/zebra, not a random other one.
- $\mathcal{L} = \mathcal{L}_{\text{GAN}}(G, D_Y) + \mathcal{L}_{\text{GAN}}(F, D_X) + 10\,\mathcal{L}_{\text{cyc}}$.
- G, F min; D_X, D_Y max.

Concept tables

Forward	Backward (also learned)
horse → zebra	zebra → horse
winter → summer	summer → winter
photo → painting style	painting → photo
sketch of a bag → photo of a bag	(unpaired collections)
Network	Map / job
———	———–
Generator G	$X \to Y$ (horse → zebra)
Generator F	$Y \to X$ (zebra → horse)
Discriminator D_Y	real **zebra** vs generated zebra $G(x)$
Discriminator D_X	real **horse** vs generated horse $F(y)$

L41: StyleGAN

Learning focus

- Name the **fine-control** failures of a vanilla $G(z)$ (hair color, smile, glasses) and define **latent-space entanglement**.
- Draw StyleGAN's pipeline: $z \xrightarrow{F} w$ (**8-layer MLP**), **affine** style $y = Aw + b$, **synthesis** from a $4 \times 4 \times 512$ constant.
- Write **AdaIN**: normalize feature maps, then restyle with (y_s, y_b).
- Place **18 convolutions**, resolution-dependent attributes, and **noise injection** (scale B).

Exam lines

- Entanglement: one z-coordinate moves many attributes because G copies correlations in p_{data}.
- StyleGAN inserts mapping F ($8\times$ FC, 512-D) so w can be traversed **per style**.
- Images grow from a **constant** $4 \times 4 \times 512$ plus **AdaIN** styles, not from feeding z into the first conv.
- AdaIN: $\hat{x} = (x - \mu)/\sigma$, then $y_s \hat{x} + y_b$.
- 18 convs, upsample every two layers; noise scaled by B for fine stochasticity.

Concept tables

You want	Can you change only that?
Butterfly **wing color**, keep pose	**No**
Face hair **brown → blonde**	No mapped latent coordinate
Remove a smile	No
Add glasses	No
Traditional GAN	StyleGAN
——————–	———-
z goes **straight** into G	z goes through a **mapping network** F first
Image synthesis starts from z	Synthesis starts from a **learned constant** tensor
Attributes mixed in z	Intermediate space w is for **styles**; attributes vary **more independently**

L42: StyleGAN 2

Learning focus

- Name the two characteristic **StyleGAN** artifacts (water-droplet blobs and phase / alignment artifacts) and where they first show up in the synthesis pyramid.
- Explain how **AdaIN** lets the generator spike a pixel so that instance-normalization statistics become a blob.
- Write the **weight modulation** and **demodulation** formulas that replace AdaIN.
- State the three architectural changes that remove progressive growing: a **fixed** generator, **skip** `toRGB` paths, and a **residual** discriminator.

Exam lines

- StyleGAN 2 exists to kill two **structural** artifacts: **water droplets** (AdaIN statistics) and **phase / alignment** (progressive growing + high-frequency upsampling).
- Droplets: replace AdaIN with $w'_{ijk} = s_i w_{ijk}$ and demodulate by the RMS of the modulated kernel plus ε.
- Phase: freeze the pyramid, let every scale skip to RGB, give the discriminator residual paths so a deep full-res critic still preserves alignment.
- Noise injection for stochastic variation is unchanged.

Concept tables

Artifact	What you see	First visible at
Water droplet	Blob-like spots on feature maps (and sometimes on the image)	From 64×64, then **stronger** through 1024×1024
Phase / alignment	Features stay locked to the **camera center** while the face turns	Same growing pyramid

Index	Meaning
—--	———
i	input channel
j	output channel
k	spatial location inside the filter
w_{ijk}	original conv weight
w'_{ijk}	modulated weight
s_i	style scale from the affine layer

L43: Practical exercise 1 — CycleGAN (advanced GAN lab)

Learning focus

- Contrast a **vanilla GAN** (one G, one D) with **CycleGAN** (two generators, two discriminators, three loss families).
- Rebuild the **PyTorch** lab: STL-10 **horse ↔ deer**, unpaired pairing, encoder–decoder G, conv D, Adam, MSE + L1.
- Read the printed metrics (parameter counts, per-epoch losses, the six-panel grid) and the reason the checkpoints are saved.

Exam lines

- CycleGAN = G_{AB}, G_{BA}, D_A, D_B + adversarial MSE + cycle L1 + identity L1.
- This notebook: PyTorch, STL-10 class **6 horses** / class **4 deer**, 400 each, 96×96, $[-1, 1]$, batch 4, 20 epochs, Adam.
- Encoder–decoder G (ReLU, tanh out); leaky-ReLU conv D; shuffle to break pairing memory.
- Save both translation directions; D_B ended slightly stronger than D_A in the demo.

Concept tables

Model	Input	Output
G_{AB}	real horse (A)	fake deer
G_{BA}	real deer (B)	fake horse
D_A	horse-shaped images	real horse vs fake horse
D_B	deer-shaped images	real deer vs fake deer

Model	Input	Output
Loss	Role	
——	——	
Adversarial (MSE in this lab)	G fools D; D separates real/fake	
Cycle-consistency (L1)	horse → fake deer → back to horse (and the other way) should reconstruct	
Identity (L1)	$G_{AB}(\text{deer}) \approx \text{deer}$, $G_{BA}(\text{horse}) \approx \text{horse}$ — stay close when the input is already in the target domain	